

PROJECT REPORT

ParkLens — Comprehensive Academic Project Report

Project Title: ParkLens — Parking Lot Occupancy & Violation Detector

Student Name: Sahil Singh

Registration Number: 24BAI10461

Course Name: Computer Vision

Course Code: CSE3010

Domain: Computer Vision, Image Processing, Smart City Systems

Execution Environment: Local Python 3.10+ (No Cloud, No Microservices, No Database Server)

Faculty: Dr. Rajneesh Kumar Patel

TABLE OF CONTENTS

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
 - 7.1 Use Case Diagram
 - 7.2 Workflow Diagram
 - 7.3 Sequence Diagram
 - 7.4 Class/Component Diagram
 - 7.5 ER Diagram
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Validation & Error Handling Matrix
12. Testing Approach
13. Challenges Faced
14. Learnings & Key Takeaways
15. Future Enhancements
16. References

1. Introduction

ParkLens is a modular, lightweight Python application designed for local execution that solves key parking lot management challenges using classical computer vision techniques.

The system processes overhead/CCTV parking lot images to:

1. Detect whether each pre-defined slot is **FREE** or **OCCUPIED** using adaptive pixel thresholding.
2. Identify rule-based **parking violations** (No-Parking zone occupation, Double Parking, Oversized Vehicle).
3. Generate **visual annotations**, CSV session logs, and **statistical analytics** (pie charts, bar charts, trend lines, text summary).

Built with OpenCV, NumPy, Matplotlib, and pytest, ParkLens requires zero external API keys, zero cloud infrastructure, and zero heavy machine-learning models.

2. Problem Statement & Scope

2.1 Background

Urban parking management relies on manual monitoring or expensive hardware sensor networks. Automated image-based monitoring provides an affordable alternative using standard camera footage.

2.2 Core Objectives

- **O1:** Automatically inspect pre-defined parking slot ROIs and determine occupancy status with confidence scores.
- **O2:** Detect rule-based parking violations using spatial contours and Intersection-over-Union (IoU) overlap metrics.
- **O3:** Log session history to CSV and compute session-level analytics with automated chart generation.
- **O4:** Maintain high modularity, comprehensive error handling, and 40 automated unit tests covering the core modules and validation scenarios.
- **O5:** Ensure clean execution across CLI, video/image frames, and analytics reporting.

2.3 Scope Constraints

- **In Scope:** Static image/video frame processing, local file I/O, CLI interface, rule-based detection, automated unit testing.

- **Out of Scope:** Cloud databases, live RTSP video server streams, deep learning model training, user authentication/authorization.
-

3. System Requirements

4 Functional Requirements (FR)

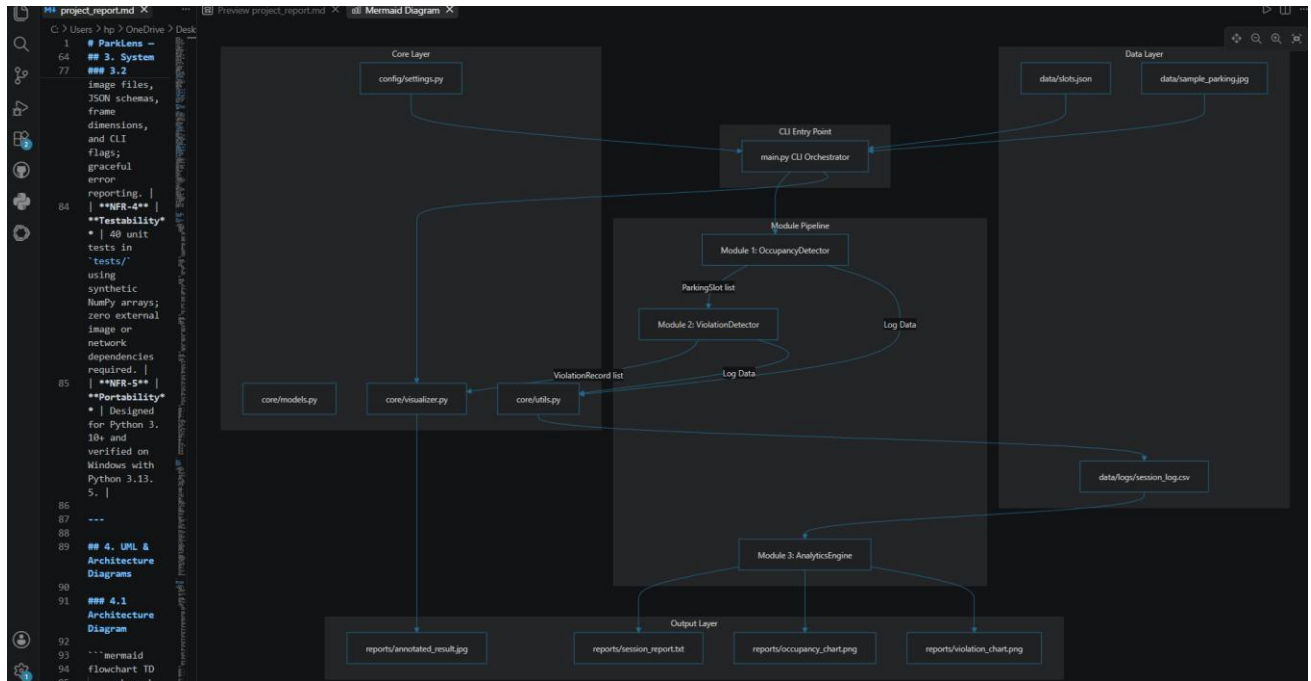
ID	Requirement	Implementation
FR-1	Slot Definition Loading	Parse ROI coordinates (x, y, w, h, slot_type) from data/slots.json with strict JSON validation.
FR-2	Occupancy Classification	Convert ROI to grayscale -> Gaussian blur -> adaptive thresholding -> classify FREE/OCCUPIED based on foreground ratio.
FR-3	Violation Detection	Evaluate 3 violation types: No-Parking Zone, Double Parking (IoU ≥ 0.25 across multiple slots), and Oversized Vehicle (Area $> 1.4 \times$ slot area).
FR-4	Visual Annotation	Draw green boxes for FREE slots, red boxes for OCCUPIED slots, orange bounding boxes for violations, and top info bar.
FR-5	Session Logging	Append per-slot and per-violation readings into data/logs/session_log.csv.
FR-6	Analytics & Reporting	Compute aggregate stats and output occupancy_chart.png, violation_chart.png, session_report.txt, and occupancy_trend.png (for multi-frame input).

5. Non-Functional Requirements (NFR)

ID	Requirement	Description & Verification
NF R-1	Modularity	3 independent functional modules (occupancy, violation, analytics) connected via clean dataclass models (core/models.py).
NF R-2	Configurability	All operational thresholds (OCCUPIED_THRESHOLD, OVERLAP_IOU_THRESHOLD, MAX_VEHICLE_AREA_RATIO) centralized in config/settings.py.
NF R-3	Validation & Safety	Comprehensive input validation on image files, JSON schemas, frame dimensions, and CLI flags; graceful error reporting.
NF R-4	Testability	40 unit tests in tests/ using synthetic NumPy arrays; zero external image or network dependencies required.
NF R-5	Portability	Designed for Python 3.10+ and verified on Windows with Python 3.13.5.

6. UML & Architecture Diagrams

6.1 Architecture Diagram

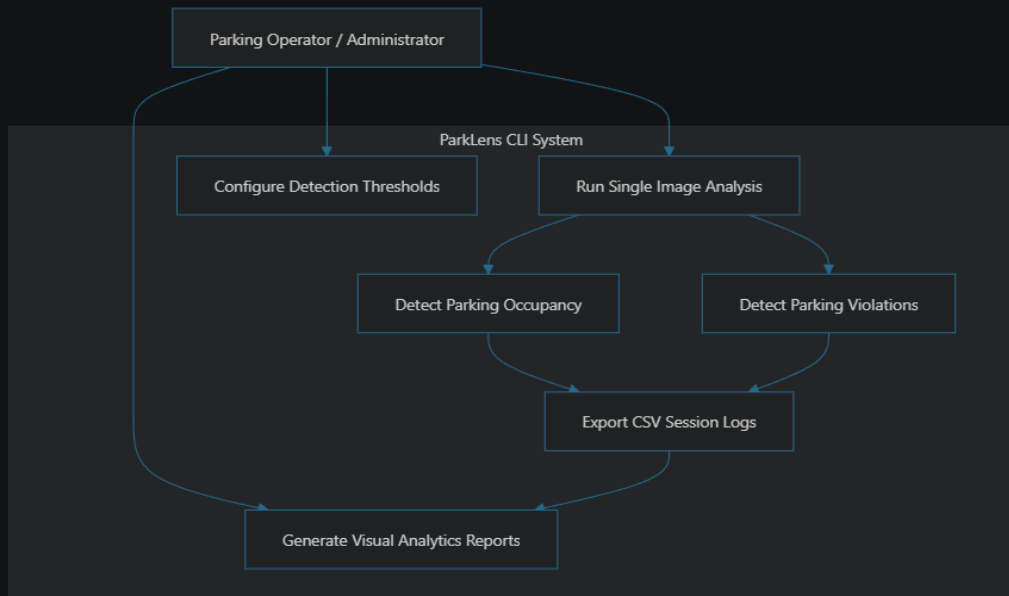


7. Design Diagrams

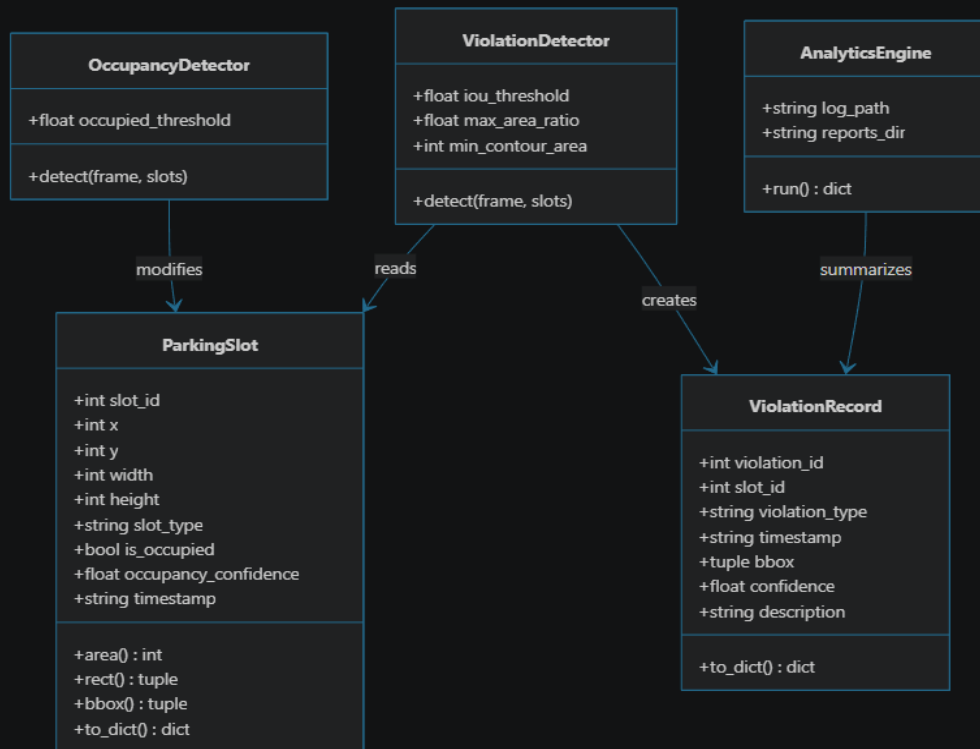
7.1 System Workflow Diagram



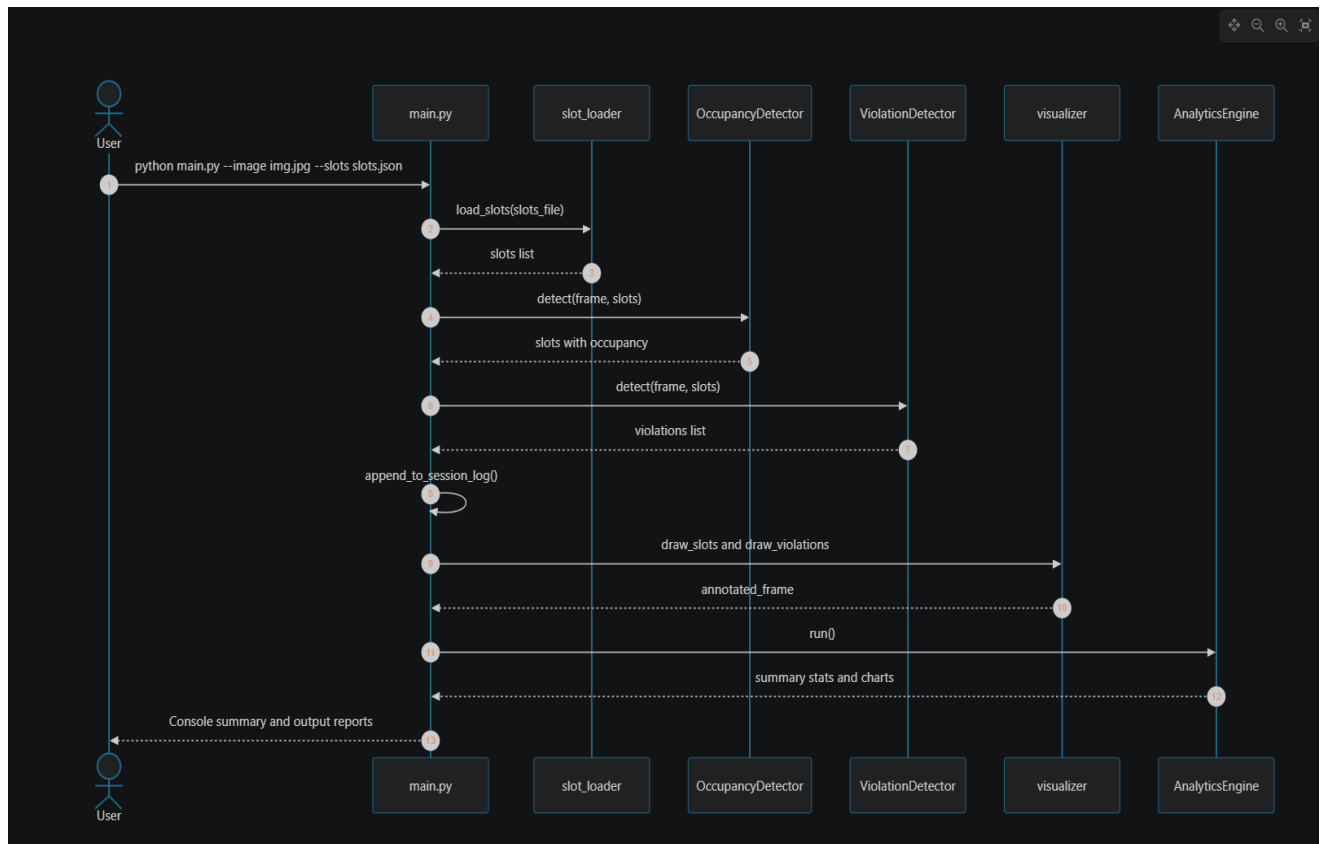
7.2 Use Case Diagram



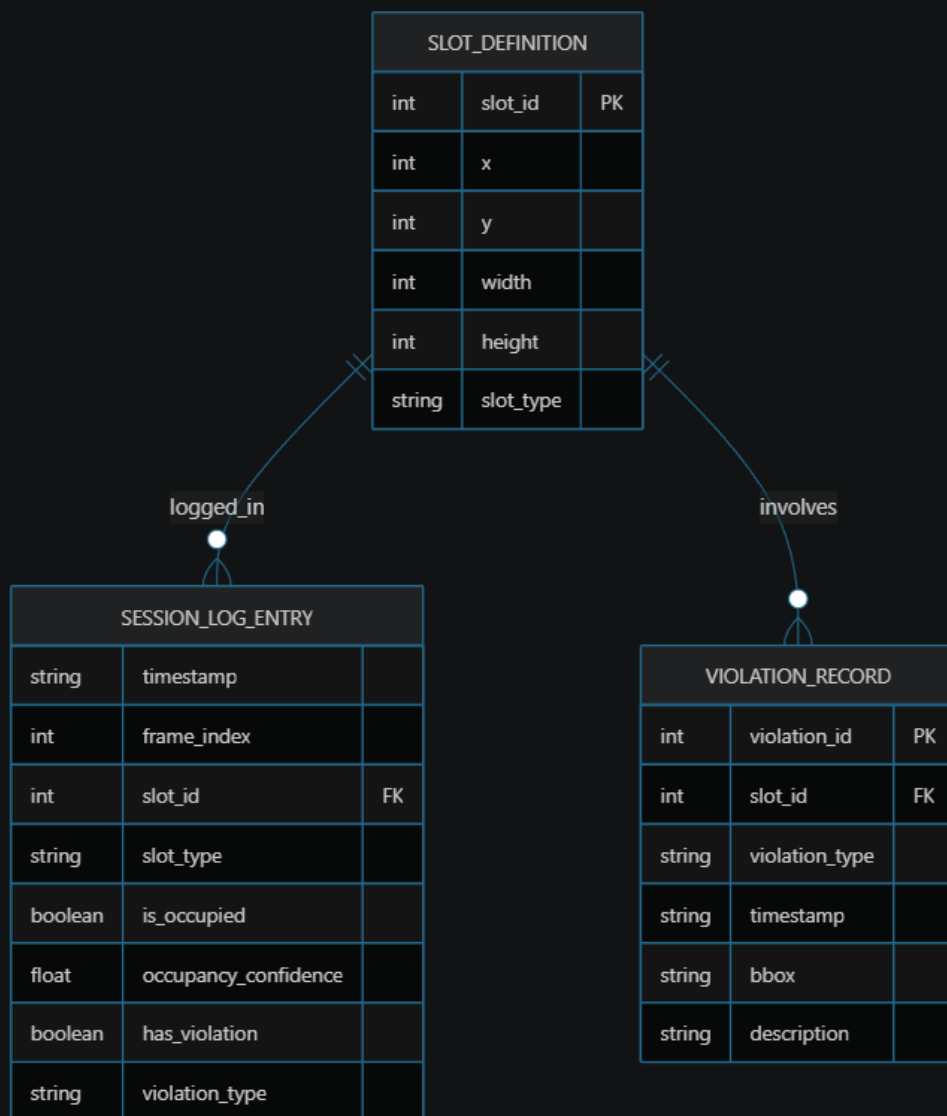
7.3 Component & Class Diagram



7.4 Sequence Diagram



7.5 Data Model / Entity Diagram



8. Design Decisions & Rationale

8.1 Classical Computer Vision Approach

ParkLens uses classical computer vision techniques such as grayscale conversion, Gaussian blur, adaptive thresholding, Canny edge detection, dilation, contour extraction, and Intersection-over-Union (IoU). This approach was selected because the project is designed as a lightweight local application and does not require a heavy machine-learning model or external inference service.

8.2 Modular Architecture

The system is divided into three independent functional modules: Occupancy Detection, Violation Detection, and Analytics. This modular design makes the system easier to understand, test, maintain, and extend.

8.3 JSON-Based Slot Configuration

Parking slot coordinates and slot types are stored in `data/slots.json` instead of being hard-coded inside the detection logic. This allows the same application to be configured for different parking layouts without changing the core implementation.

8.4 CSV-Based Session Logging

Session results are stored in `data/logs/session_log.csv`. CSV was selected because it is simple, human-readable, easy to inspect, and sufficient for the project's local, submission-focused requirements without introducing a database server.

8.5 Rule-Based Violation Detection

Violations are identified using explicit spatial rules for No-Parking Zone, Double Parking, and Oversized Vehicle. This keeps the implementation transparent and allows each rule

to be independently tested.

8.6 Local Execution

ParkLens runs locally using Python, OpenCV, NumPy, Matplotlib, and pytest. No cloud infrastructure, external API keys, microservices, or database server is required.

This keeps deployment simple and reduces external dependencies.

9. Module Implementation Details

Module 1: Occupancy Detection (modules/occupancy/)

- **slot_loader.py**: Parses slots.json into dataclass instances with schema validation.
- **detector.py**: Crops ROI per slot -> converts to grayscale -> Gaussian blur (5x5) -> adaptive Gaussian thresholding (block_size=11, C=2) -> counts non-zero pixels. If ratio ≥ 0.35 , marks slot as **OCCUPIED**.

Module 2: Violation Detection (modules/violation/)

- **detector.py**: Uses Canny edge detection + dilation + contour extraction to locate vehicle bounding boxes.
- Evaluates 3 rule-based violations:
 1. **No-Parking Zone**: Slot marked no_parking and is occupied.
 2. **Double Parking**: Vehicle bounding box overlaps ≥ 2 occupied slots with IoU ≥ 0.25 .
 3. **Oversized Vehicle**: Vehicle contour area $> 1.4x$ slot area.

Module 3: Analytics Engine (modules/analytics/)

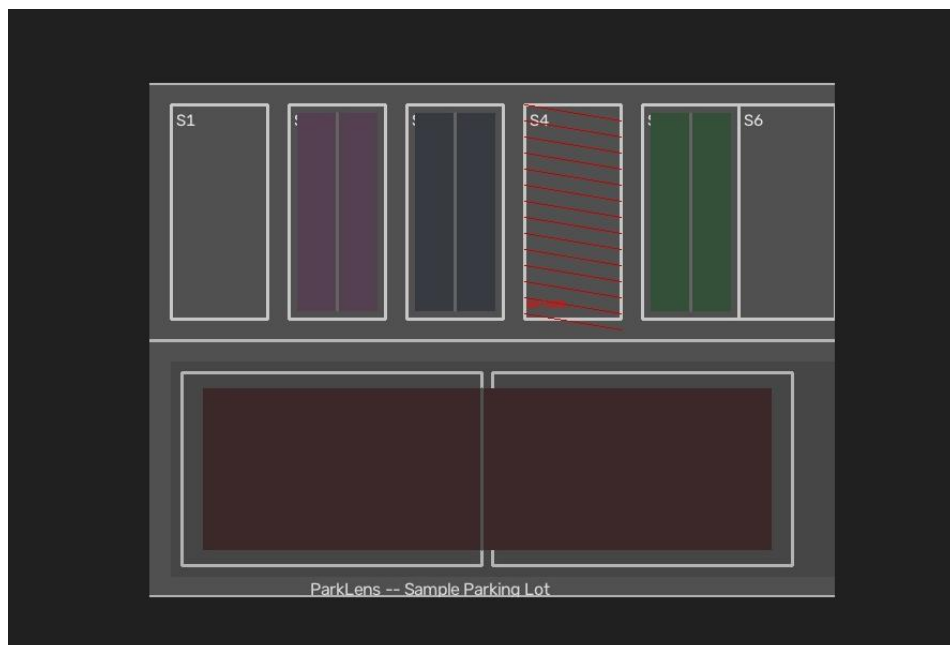
- **engine.py**: Aggregates CSV log entries from data/logs/session_log.csv.
- Generates:
 - occupancy_chart.png: Matplotlib pie chart (Free vs. Occupied).
 - violation_chart.png: Matplotlib bar chart (Violations by Type).

- occupancy_trend.png: Line chart over time (for multi-frame inputs).
 - session_report.txt: Structured plain text report.
-

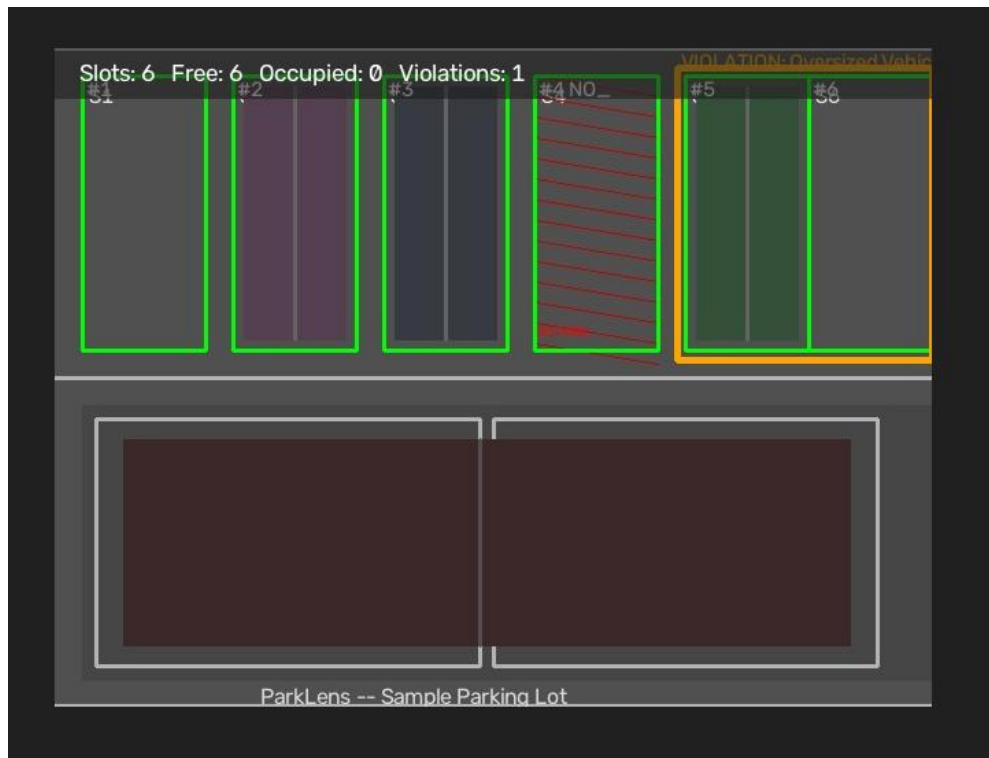
10. Artifact & Screenshot Checklist

The following output artifacts are generated upon execution and available in the workspace.

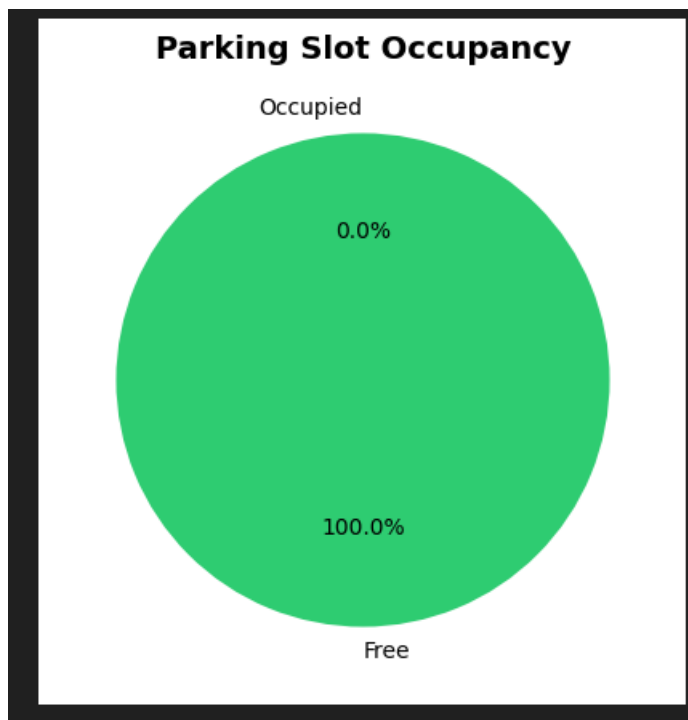
10.1 Input Parking Image



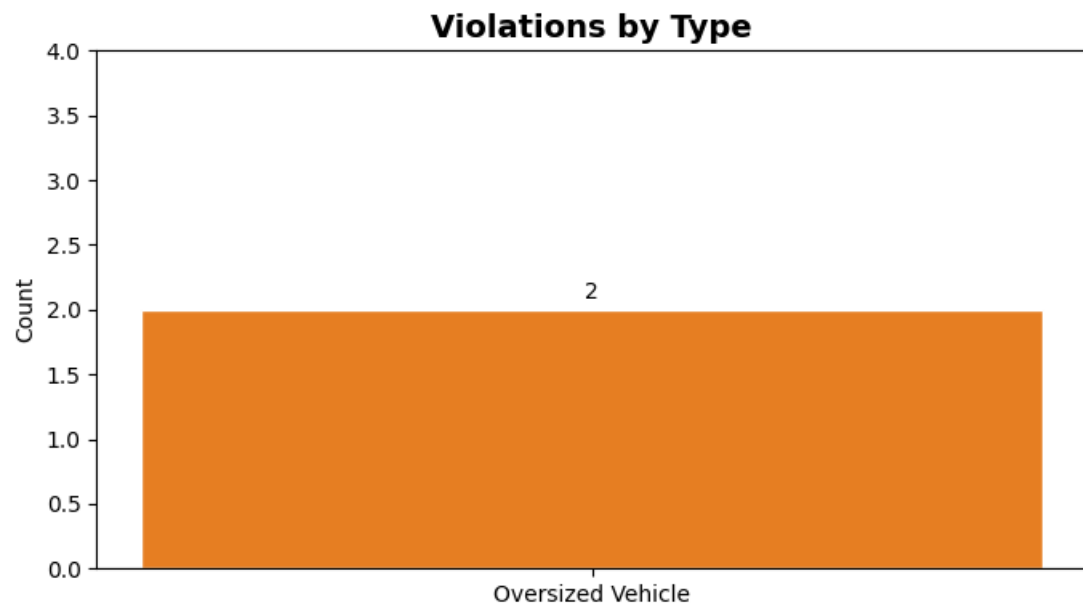
10.2 Annotated Detection Result



10.3 Occupancy Analysis



10.4 Violation Analysis



11. Validation & Error Handling Matrix

Component	Error Scenario	Handling Mechanism	User Output
CLI / Input	Missing or invalid image path	utils.validate_image_path() checks existence and extension.	[ERROR] Image file not found
CLI / Input	Malformed slots .json	utils.validate_slots_data() checks top-level keys & slot schema.	[ERROR] Slot at index N is missing required keys
Module 1	Empty / None frame passed	OccupancyDetector._validate_frame() checks NumPy array type & size.	Raises ValueError with clear message
Module 1	Slot coordinates outside frame bounds	OccupancyDetector._crop_roi() clamps bounding box to [0, width] and [0, height].	Gracefully processes clamped ROI
Module 3	Session CSV log missing	AnalyticsEngine._load_csv() checks path existence.	[ERROR] Session log not found
Module 3	Empty CSV file (header only)	Checks row count post-header reading.	[ERROR] Session log CSV contains no data rows
Console Output	Windows terminal non-ASCII encoding	Output sanitized to standard ASCII (->, x, sq).	Clean, ungarbled console output

12. Testing & Verification Results

12.1 Unit Test Suite Execution

Executed with `python -m pytest tests/ -v:`

```
===== test session starts
```

```
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
```

```
collected 40 items
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_occupancy_stats_correct PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_violation_count_correct PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_no_violations_in_clean_session PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_multiple_frames_trend_computed PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_occupancy_chart_created PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_violation_chart_created PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_text_report_created PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_trend_chart_created_for_multi_frame PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_trend_chart_not_created_for_single_frame PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_raises_when_log_missing PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_raises_when_log_empty PASSED
```

```
tests/test_analytics.py::TestAnalyticsEngine::test_stats_dict_has_all_expected_keys PASSED
```

```
tests/test_occupancy.py::TestOccupancyDetector::test_free_slot_on_empty_tarmac PASSED
```

```
tests/test_occupancy.py::TestOccupancyDetector::test_occupied_slot_with_dark_vehicle PASSED
```

```
tests/test_occupancy.py::TestOccupancyDetector::test_confidence_is_between_0_and_1 PASSED
```

```
tests/test_occupancy.py::TestOccupancyDetector::test_timestamp_is_set PASSED
```

```
tests/test_occupancy.py::TestOccupancyDetector::test_multiple_slots_detected_independently PASSED
```

```
tests/test_occupancy.py::TestOccupancyDetector::test_slot_partially_outside_frame_is_handled PASSED
```

```
tests/test_occupancy.py::TestOccupancyDetector::test_threshold_zero_marks_everything_occupied PASSED
```

tests/test_occupancy.py::TestOccupancyDetector::test_threshold_one_marks_everything_free PASSED

tests/test_occupancy.py::TestOccupancyDetector::test_raises_on_none_frame PASSED

tests/test_occupancy.py::TestOccupancyDetector::test_raises_on_empty_frame PASSED

tests/test_occupancy.py::TestOccupancyDetector::test_raises_on_non_array_frame PASSED

tests/test_occupancy.py::TestOccupancyDetector::test_returns_same_list_object PASSED

tests/test_occupancy.py::TestSlotLoader::test_load_valid_slots_file PASSED

tests/test_occupancy.py::TestSlotLoader::test_raises_on_missing_file PASSED

tests/test_occupancy.py::TestSlotLoader::test_raises_on_invalid_json PASSED

tests/test_occupancy.py::TestSlotLoader::test_raises_on_missing_slots_key PASSED

tests/test_occupancy.py::TestSlotLoader::test_raises_on_empty_slots_array PASSED

tests/test_occupancy.py::TestSlotLoader::test_raises_on_slot_missing_required_key PASSED

tests/test_violation.py::TestViolationDetector::test_no_parking_zone_occupied_raises_violation PASSED

tests/test_violation.py::TestViolationDetector::test_no_parking_zone_free_no_violation PASSED

tests/test_violation.py::TestViolationDetector::test_regular_occupied_slot_no_no_parking_violation PASSED

tests/test_violation.py::TestViolationDetector::test_violation_record_has_required_fields PASSED

tests/test_violation.py::TestViolationDetector::test_no_violations_on_all_free_regular_slots PASSED

tests/test_violation.py::TestViolationDetector::test_iou_overlapping_boxes PASSED

tests/test_violation.py::TestViolationDetector::test_iou_non_overlapping_boxes PASSED

tests/test_violation.py::TestViolationDetector::test_iou_partial_overlap PASSED

tests/test_violation.py::TestViolationDetector::test_iou_zero_area_box PASSED

tests/test_violation.py::TestViolationDetector::test_violation_record_to_dict PASSED

===== 40 passed
=====

12.2 Integration Smoke Test Output

Executed with python main.py --image data/sample_parking.jpg --slots data/slots.json --no-display:

[ParkLens] Image : data/sample_parking.jpg

[ParkLens] Slots : 6 defined

[ParkLens] Output : reports

[Module 1 -- Occupancy] Total=6 Occupied=0 Free=6

Slot # 1 [regular] FREE (confidence=14.77%)

Slot # 2 [regular] FREE (confidence=18.04%)

Slot # 3 [regular] FREE (confidence=21.80%)

Slot # 4 [no_parking] FREE (confidence=27.62%)

Slot # 5 [regular] FREE (confidence=25.94%)

Slot # 6 [regular] FREE (confidence=14.77%)

[Module 2 -- Violations] 1 violation(s) found

Slot #5 oversized_vehicle -- Vehicle area 39618px sq exceeds 1.4x slot area 18000px sq.

[ParkLens] Annotated image saved -> reports\annotated_result.jpg

[Module 3 -- Analytics]

=====

ParkLens -- Session Analytics Report

Generated: 2026-09-18T18:13:44

=====

OCCUPANCY SUMMARY

Total parking slots : 6

Total slot readings : 12

Occupied readings : 0

Free readings : 12

Avg occupancy : 0.00%

VIOLATION SUMMARY

Total violations : 2

* Oversized Vehicle : 2

TOP OFFENDING SLOTS

* Slot #5 : 2 violation(s)

=====

[ParkLens] Reports saved -> reports

13. Challenges Faced

13.1 Occupancy Detection from Image Data

One of the main challenges was determining whether a parking slot was occupied using only image information. Changes in brightness, shadows, and the appearance of the parking surface can affect the pixel values inside a slot. To handle this, the occupancy module uses grayscale conversion, Gaussian blur, and adaptive thresholding before calculating the foreground ratio.

13.2 Parking Slot Configuration

Different parking areas can have different slot positions and dimensions. Keeping the coordinates directly inside the source code would make the application difficult to configure. Therefore, parking slot coordinates and slot types were stored separately in `data/slots.json`.

13.3 Rule-Based Violation Detection

Detecting a violation requires additional spatial analysis after occupancy detection. For example, double parking requires checking whether a vehicle overlaps multiple occupied slots. Oversized vehicle detection requires comparing the detected vehicle area with the corresponding parking slot area.

13.4 Input Validation

The application can receive invalid inputs such as a missing image, malformed JSON file, empty frame, invalid slot coordinates, or missing session logs. Validation checks were therefore required at different stages of the application to provide meaningful error messages and avoid unexpected execution failures.

13.5 Maintaining a Lightweight Application

The project was designed to work locally without cloud infrastructure, external APIs, microservices, or a database server. Because of this, configuration and session information were handled using JSON and CSV files. This kept the overall system simple and suitable for local execution.

14. Learnings & Key Takeaways

The development of ParkLens provided practical experience in building a small computer vision application from input processing to final analytics.

The major learnings from the project include:

- Understanding the basic image preprocessing pipeline used in computer vision.
- Working with grayscale conversion and Gaussian filtering.
- Understanding adaptive thresholding and its use in occupancy classification.
- Working with Region of Interest (ROI) based image processing.
- Using Canny edge detection and contour extraction for identifying vehicle regions.
- Understanding bounding boxes and spatial relationships between detected regions
- Applying Intersection-over-Union (IoU) for checking overlap between vehicle and parking-slot regions.
- Designing a modular Python application where different tasks are handled by separate modules.
- Using JSON files to store configurable parking-slot information.
- Using CSV files for storing session-level detection information.
- Generating analytical charts using Matplotlib.
- Creating automated tests using pytest.
- Handling invalid inputs and edge cases through validation.
- Integrating multiple processing stages into a single command-line application.

One of the important takeaways from the project was that implementing the detection logic is only one part of a complete software project. Proper input validation, modular structure, testing, logging, documentation, and output generation are also important for making the application usable and maintainable.

15. Future Enhancements

The current version of ParkLens processes configured parking slots from locally available images. The following improvements can be considered for future versions.

15.1 Real-Time CCTV Processing

The system can be extended to process continuous CCTV or video input instead of only processing individual images. This would allow parking occupancy and violations to be monitored continuously.

15.2 Deep Learning-Based Vehicle Detection

A deep learning-based object detection model could be integrated into the violation detection module. Such a model could provide more robust vehicle detection in scenes with different backgrounds, lighting conditions, and vehicle shapes.

15.3 Automatic Parking Slot Detection

The current implementation uses parking-slot coordinates provided through `data/slots.json`. A future version could automatically identify parking spaces from the input image, reducing the need for manual slot configuration.

15.4 Web-Based Dashboard

A web-based dashboard could be added to display the number of available and occupied slots, detected violations, and historical analytics through a graphical interface.

15.5 Database Integration

The current application uses CSV files for session logging. For a larger deployment, the logs could be stored in a database to support long-term storage, searching, filtering, and analysis of parking data.

15.6 Advanced Analytics

Additional analytics could be introduced to identify peak occupancy periods, frequently used parking slots, violation frequency, and daily or weekly parking trends.

15.7 Improved Detection Conditions

Future versions could be tested and improved for different camera angles, shadows, weather conditions, vehicle sizes, and parking environments to make the detection process more reliable.

16. References

1. Python Software Foundation. Python Documentation.
<https://docs.python.org/3/>
2. OpenCV. Open Source Computer Vision Library Documentation.
<https://docs.opencv.org/>
3. NumPy. NumPy Documentation.
<https://numpy.org/doc/>
4. Matplotlib. Matplotlib Documentation.
<https://matplotlib.org/stable/>
5. pytest. pytest Documentation.
<https://docs.pytest.org/>
6. VITyarthi Project Requirements and Submission Guidelines.
Course-provided project documentation.